



Flipkart GRiD 8.0

Student Preparation Brief — Round-by-Round Analysis

Basic Details — Flipkart GRiD 8.0

- What it is: India's largest engineering challenge, now positioned as a "foundational AI accelerator" (8th edition).
- Tracks: Students can apply to any/all of 3 tracks — Software Development, AI Engineering, Data Science. Shortlisting into a specific track happens based on performance.
- Eligibility — Colleges: All engineering colleges across India.
 - Eligibility — Degrees: B.Tech, B.E., M.Tech, M.E., M.S., PhD, Dual/Integrated.
 - Eligibility — Graduation year: 2027–2030 (Bachelor's/Master's), 2027–2031 (PhD/Dual).
- Outcomes: Full-Time SDE roles & Winter Internships (4th-year students); Summer Internships (3rd-year students).
- Certificates for all Round 1, 2, 3 qualifiers; Top 50 get a platform-wide spotlight.
- Cost: Completely free — no fees at any stage. Any payment demand is a scam.
- Format: Fully remote/online, laptop/desktop only (no mobile devices).
- Finals location: Flipkart HQ, Bangalore — travel & stay covered by Flipkart for finalists.
- Registration: Individual only; accurate data is mandatory — incorrect info leads to instant disqualification.

Round 1 — Screening

Type: Profile/resume evaluation — no test.

Areas evaluated: Academic consistency (CGPA trend), non-academic achievements (coding ratings, hackathons, certifications, GitHub activity), overall profile strength.

Insights from candidate reviews

- Consistency over peaks — a steady 7.5–8 CGPA across semesters is viewed better than a dip-and-recover pattern.
- Specific, verifiable claims beat generic skill lists ("Solved 300+ problems on LeetCode, rating 1650" > "Good at DSA").
- Data accuracy matters — incorrect submitted info leads to instant disqualification per the official FAQ.

Round 2 — Coding Assessment (90 minutes, proctored, ~3 problems)

Structure: Typically 1 hard + 2 medium DSA problems; sometimes preceded by a CS-fundamentals MCQ section (OS, DBMS, OOPs, networking).

Topics & LeetCode references

Topic	Sample Question Style	LeetCode Reference
Arrays & Subarrays	Subarray sum, max sum subarray	LC 560, LC 53
Strings	Anagram grouping, remove duplicates, pattern matching	LC 49, LC 1047, LC 28
Two Pointers	Palindrome checks / partitioning	LC 125, LC 5, LC 11



Topic	Sample Question Style	LeetCode Reference
Sliding Window	Longest substring without repeats	LC 3, LC 209
Hashing	Two sum variants, frequency counting	LC 1, LC 387
Stack / Queue	Valid parentheses, next greater element	LC 20, LC 496
Heap	Kth largest, top-k frequent, simulation merging	LC 215, LC 347, LC 1046
Greedy	Minimum jumps to reach end	LC 45, LC 55
Graphs (over-represented)	Grid traversal, infection spread, shortest path with constraints	LC 994, LC 200, LC 787
Dynamic Programming (over-represented)	Array/string optimization, memo vs tabulation	LC 322, LC 300, LC 62/63
Trees	Traversals, height, LCA	LC 104, LC 236, LC 102
Linked Lists	Reversal, cycle detection, merging	LC 206, LC 141, LC 21
Backtracking	Permutations, subsets	LC 46, LC 78
Bit Manipulation	Single number, count set bits	LC 136, LC 191
Binary Search	Rotated array, search-on-answer style	LC 33, LC 34

Real previous-year OA questions reported by candidates

- Pick 2 array elements, add their sum to a result, reinsert (sum - 1); repeat until one remains (simulation/greedy — akin to LC 1046).
- Shortest route between two cities where up to k connected edges can be zeroed out (modified Dijkstra/BFS with an extra state dimension — akin to LC 787).
- Find the smallest missing positive integer, with multiple approaches/complexities expected (LC 41).
- Days until an entire grid is "infected" from initial infected cells (LC 994 family).
- A non-trivial binary-search-based problem, reported as harder to reason through than it first appears.
- An "advanced string matching" question — KMP/Rabin-Karp fundamentals worth revising.

Strategy insights from Glassdoor / candidate reviews

- Speed is a real signal, not just correctness — fast, accurate finishers reported getting interview calls.
- Scoring is based on test cases cleared vs. cutoff — always submit a working brute force before optimizing, to lock in partial credit.
- Time-box: ~25–30 minutes per problem with a buffer; scan all 3 questions first, attempt the easiest first.
- Test edge cases (empty array, single element, negatives) before submitting.
- Don't switch programming languages mid-test — costs time, earns nothing.

Round 3 — Case Study Code

Type: A real Flipkart business problem → build a system-level technical solution or tool → submit code for review.



Clarity — Is Round 3 web development?

No. This is a backend logic / system-design round, not a frontend/UI-building test.

Past problems: cart optimization, delivery ETA estimation, search indexing, recommendation logic, wishlist/price-drop notifications, inventory management — all logic/architecture heavy.

Judges have repeatedly said a clear, working, well-explained solution beats a complicated or polished one.

A UI is optional and useful only if it helps explain the solution — it is not what's graded.

Track matters: Software Dev → APIs/data models/logic/concurrency. AI Engineering → a working ML/AI pipeline or feature. Data Science → analysis/model/insights. None require a built website.

Sample problem styles

- Simplified ticket-booking system (seat locking, no double-booking).
- Inventory/warehouse stock tracking with restock triggers.
- Flash-sale checkout flow (prevent overselling).
- URL shortener / rate limiter API (reported in adjacent interview rounds too).
- Rule-based recommendation engine.

Insights

- Concurrency-aware design (locking, rate limiting, overselling prevention) is over-represented across this round and later interview rounds — worth dedicated conceptual prep.
- Code should be clean and modular, readable by a reviewer skimming many submissions.

Round 4 — National Finals (Hackathon, teams of 4)

Type: Top 48 (16/track) at Flipkart HQ, Bangalore — build a live working prototype and present to leadership.

Clarity — This is where building/UI actually matters

Unlike Round 3, Round 4 is a live demo-based hackathon — teams genuinely build and present a working prototype (which can include a UI/frontend if relevant to the pitch).

This is the stage to invest polish and demo-readiness, not Round 3.

Insights from past winners / reviews

- Teams that scope down aggressively and ship something working beat teams with ambitious-but-broken builds.
- Clear role division on day one (frontend / backend / data-ML / pitch) avoids last-hour chaos.
- Judges respond better to solutions tied to a specific, real user problem than generic pitches.
- Candidates get grilled on their own project/architecture choices live — not just asked to demo.

Track-specific pointers

- Software Development → justify DB/API design trade-offs live.
- AI Engineering → have a backup (pre-recorded clip) in case live inference fails.
- Data Science → explain why a metric/model was chosen, not just show a dashboard.



Presentation prep

- Practice a tight 3-minute pitch: problem → solution → demo → impact. Judges see many teams back-to-back.

Quick Reference Summary

Round	Format	Key Focus
1 — Screening	Profile-based, no test	Academic consistency + achievements
2 — Coding Assessment	90 min proctored, ~3 problems	DSA (Graphs & DP over-represented), speed + partial credit
3 — Case Study Code	Take-home business problem	Backend logic/system design, NOT web dev
4 — National Finals	Team hackathon at Flipkart HQ	Working prototype, live demo, presentation